Programming Assignment #2
Assigned 2/8/2026
Due on Canvas 2/22/2026 at 11:59PM
100 points

The learning goals of this assignment are as follows.
Students will learn to:

- Represent problem states symbolically in Prolog
- Write **safe-state constraints**
- Generate valid successor states (moves)
- Implement a **search procedure** (DFS with cycle checking and BFS for shortest solution)
- Return a **solution path** and report its length

STATEMENT OF THE PROBLEM
There are **3 missionaries** and **3 cannibals** on the **left bank** of a river. A boat can carry **1 or 2 people** across. If, on either bank, the number of **cannibals exceeds missionaries**, the missionaries get eaten (unless there are **0 missionaries** on that bank).
Goal: move everyone safely to the **right bank**.

STATE REPRESENTATION
Represent the state as a list `[ML,CL, Side]`
were

- `ML` = missionaries on left bank from 0 to 3
- `CL` = cannibals on left bank from 0 to 3
- `Side` indicates where the boat is [`left, right`]

The initial state is represented as: `start([3,3, left]).`
The goal state is represented as: `goal([0,0,right]).`

Define the safety constraints
Define `safe([ML,CL,_])` to convey the safety rules:

- Values must stay in range 0 to 3
- On each bank:
  - either missionaries are 0, or missionaries ≥ cannibals.

Note that right bank has 3 minus the number on the left bank: `MR = 3-ML`, `CR = 3-CL`.

MOVES

A move transfers **1 or 2 people**:

Possible boat passenger combinations are as follows: (2M,0C), (0M,2C), (1M,0C), (0M,1C), (1M,1C)

Define:

- `move([ML,CL,left], [ML2,CL2,right], Action).`
- `move([ML,CL,right],[ML2,CL2,left], Action).`

Where `Action` is something like

`boat(M,C)` meaning M missionaries and C cannibals moved.

Constraints:

- The departing bank must have enough people to send.
- The arriving state must satisfy `safe(State).`

SEARCH 1: DFS with cycle checking

Must define the predicate: `solve_dfs(Path),` which returns the path as a list of states from initial to goal states. Must avoid loops by tracking visited states.

SEARCH 2: BFS for shortest path

Must define the predicate**:** `solve_bfs(Path),` which returns the path as a list of states from initial to goal states. Must avoid loops by tracking visited states.

Output requirements

For each search (DFS or BFS), define the predicate `run(Algorithm),` which when called, it should:

1. Find a solution path (DFS or BFS)
2. Print the path state-by-state
3. Print the number of crossings (path length-1)
4. Optionally, print the actions taken between states

Required test cases

Include queries like:

1. `?- safe([3,3,left]).` should succeed
2. `?- safe([1,2,left]).` should fail (left bank: 1M < 2C)

3. `?- move([3,3,left], S2, A).` should generate only legal safe successors
4. `?- solve_bfs(P).` should return P, a path that ends at state `[0,0,right]`
5. `?- solve_dfs(P).` should return P, a path that ends at state `[0,0,right]`
6. Compare the length of the paths produced by DFS and BFS
7. Path validity checker every consecutive pair in P must be a `move`

DELIVERABLES

1. `mc.xx` (main program)
2. Comment describing:
    - state format
    - how to run
    - which search they implemented
3. `tests.xx` with sample queries

Grading rubric (100 points)

- State representation & initial state/goal state (10)
- Define the safety predicate correctly (10)
- Move generator correct & complete (10)
- DFS works, terminates, avoids cycles (20)
- BFS works, terminates, avoids cycles (20)
- DFS produces a valid solution path (10)
- BFS produces a valid solution path (10)
- Readable output + documentation/tests (10)